

File Edit Selection View Go Run Terminal Help

python_test

EXPLORER

- PYTHON_TEST
 - __pycache__
 - .vscode
 - settings.json
 - New folder
 - annotated_image.jpg
 - black.jpg
 - clock.png
 - create_test_image_day3.py
 - create_test_image.py
 - DAY-01 Program Outputs.pdf
 - DAY-02 Program outputs.pdf
 - DAY-03 Program Outputs.pdf
 - DAY-04 Program Outputs.pdf
 - day5_transformations.py
 - drawing_basics.py
 - drawing.py
 - exercise1.day3.py
 - exercise1.house.py
 - exercise1.py
 - exercise2.day3.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.chessboard.py
 - exercise3.day3.py
 - exercise3.py
 - exercise4.annotations.py
 - exercise4.day3.py
 - exercise5.clock.py
 - getting_and_setting.py
 - gradient.png
 - house.png
 - image.py
- OUTLINE
- TIMELINE

day5_transformations.py

```
254 img1 = imutils.resize(image, width=300)
255 img1 = imutils.crop(img1, 0, 200, 0, 300) # E
256 size
257 # 2. Flip horizontally
258 img2 = imutils.flip(image, 'horizontal')
259 img2 = imutils.resize(img2, width=300)
260 # 3. Rotate 90 degrees
261 img3 = imutils.rotate(image, 90)
262 img3 = imutils.resize(img3, width=300)
263 # 4. Crop the center
264 img4 = imutils.crop(image, height//3, 2*height//3, width//3, 2*width//3)
265 img4 = imutils.resize(img4, width=300)
266 # Place images on collage
267 collage[0:200, 0:300] = img1
268 collage[0:200, 350:650] = img2
269 collage[250:450, 0:300] = img3
270 collage[250:450, 350:650] = img4
271 # Add labels
272 cv2.putText(collage, "Original", (100, 30),
273 cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
274 cv2.putText(collage, "Horizontal Flip", (430, 30),
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Loaded image: 350 x 228 pixels

[Section 2] Translation - Shifting the image...

Theory: Translation moves the image along X and Y axes. Transformation Matrix: $\begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \end{bmatrix}$ where tx=right/left, ty=up/down

--- Method 1: Manual translation ---

--- Method 2: Using our convenience function ---

--- Edge Effects ---

Observation: Empty areas are filled with black (0)

[Section 3] Rotation - Turning the image...

Theory: Rotation spins the image around a center point. Positive angle = counter-clockwise, Negative = clockwise

--- Manual Rotation ---



File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - __pycache__
 - .vscode
 - settings.json
 - New folder
 - annotated_image.jpg
 - black.jpg
 - clock.png
 - create_test_image_day3.py
 - create_test_image.py
 - DAY-01 Program Outputs.pdf
 - DAY-02 Program outputs.pdf
 - DAY-03 Program Outputs.pdf
 - DAY-04 Program Outputs.pdf
 - day5_transformations.py
 - drawing_basics.py
 - drawing.py
 - exercise1.day3.py
 - exercise1.house.py
 - exercise1.py
 - exercise1.thumbnail.py**
 - exercise2.day3.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.chessboard.py
 - exercise3.day3.py
 - exercise3.py
 - exercise4.annotations.py
 - exercise4.day3.py
 - exercise5.clock.py
 - getting_and_setting.py
 - gradient.png
 - house.png
- OUTLINE
- TIMELINE

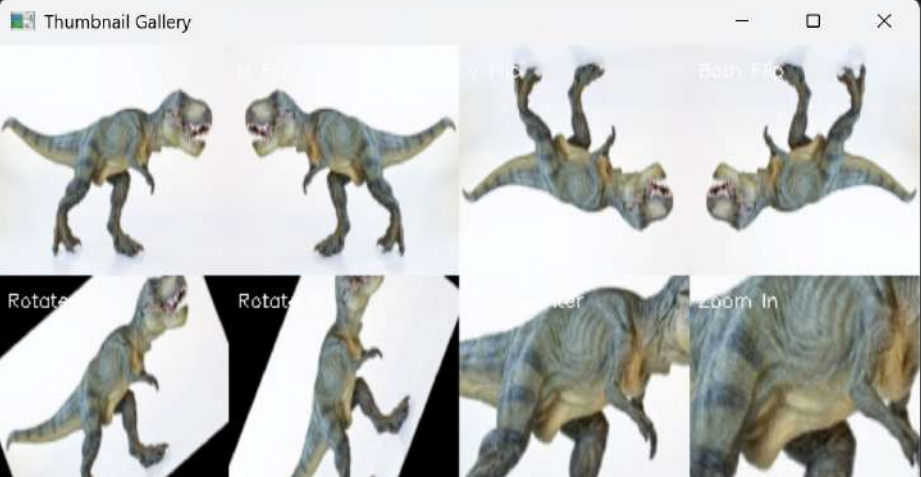
exercise1.thumbnail.py

```
20 verticalai', width=thumb_size)),
21 ("Both Flip", lambda img:
22 imutils.resize(imutils.flip(img, 'both'), width=thumb_size)),
23 ("Rotate 30", lambda img:
24 imutils.resize(imutils.rotate(img, 30), width=thumb_size)),
25 ("Rotate 60", lambda img:
26 imutils.resize(imutils.rotate(img, 60), width=thumb_size)),
27 ("Crop Center", lambda img: imutils.resize(
28 imutils.crop(img, img.shape[0]//4, 3*img.shape[0]//4,
29 img.shape[1]//4, 3*img.shape[1]//4),
30 width=thumb_size)),
31 ("Zoom In", lambda img: imutils.resize(
32 imutils.crop(img, img.shape[0]//3, 2*img.shape[0]//3,
33 img.shape[1]//3, 2*img.shape[1]//3),
34 width=thumb_size))
35 ]
36 for i, (name, transform) in enumerate(transformations):
37     row = i // 4
38     col = i % 4
39     thumb = cv2.resize(transform(image), (thumb_size, thumb_size))
40     v_start = row * thumb_size
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python exercise1.thumbnail.py --image terex.png

Thumbnail Gallery



The Thumbnail Gallery window displays a 2x4 grid of images showing different transformations of a dinosaur image. The transformations are: 1. Original image, 2. Both Flip (horizontal and vertical), 3. Rotate 30 degrees, 4. Rotate 60 degrees, 5. Crop Center, 6. Zoom In, 7. Crop Center, 8. Zoom In. The images are arranged in two rows of four.

Python 3.14.5 Python 3.14 (64-bit)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - __pycache__
 - .vscode
 - settings.json
 - New folder
 - annotated_image.jpg
 - black.jpg
 - clock.png
 - create_test_image_day3.py
 - create_test_image.py
 - DAY-01 Program Outputs.pdf
 - DAY-02 Program outputs.pdf
 - DAY-03 Program Outputs.pdf
 - DAY-04 Program Outputs.pdf
 - day5_transformations.py
 - drawing_basics.py
 - drawing.py
 - exercise1.day3.py
 - exercise1.house.py
 - exercise1.py
 - exercise1.thumbnail.py
 - exercise2.day3.py
 - exercise2.panorama.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.chessboard.py
 - exercise3.day3.py
 - exercise3.py
 - exercise4.annotations.py
 - exercise4.day3.py
 - exercise5.clock.py
 - getting_and_setting.py
 - gradient.png
- OUTLINE
- TIMELINE

exercise2.panorama.py

```
1 # exercise_2_panorama_prep.py
2 import numpy as np
3 import cv2
4 import imutils
5 # Create two shifted versions of an image to simulate panorama stitching
6 image = cv2.imread("test_image.png") # Use your image
7 # Create left and right shifted versions
8 left_shift = imutils.translate(image, -80, 0)
9 right_shift = imutils.translate(image, 80, 0)
10 # Create overlay to show overlap
11 overlay = image.copy()
12 overlay[0:image.shape[0], 0:80] = left_shift[0:image.shape[0], image.shape[1]-80:image.shape[1]]
13 cv2.putText(overlay, "Overlap Region", (10, 50),
14 cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)
15 # Create side-by-side comparison
16 comparison = np.hstack([left_shift, image, right_shift])
17 cv2.imshow("Panorama Preparation (Left | Original | Right)",
18 comparison)
```

Panorama Preparation (Left | Original | Right)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - __pycache__
 - .vscode
 - settings.json
 - New folder
 - annotated_image.jpg
 - black.jpg
 - clock.png
 - create_test_image.py
 - create_test_image.py
 - DAY-01 Program Outputs.pdf
 - DAY-02 Program Outputs.pdf
 - DAY-03 Program Outputs.pdf
 - DAY-04 Program Outputs.pdf
 - day5_transformations.py
 - drawing_basics.py
 - drawing.py
 - exercise1.day3.py
 - exercise1.house.py
 - exercise1.py
 - exercise1.thumbnail.py
 - exercise2.day3.py
 - exercise2.panorama.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.animation.py
 - exercise3.chessboard.py
 - exercise3.day3.py
 - exercise3.py
 - exercise4.annotations.py
 - exercise4.day3.py
 - exercise4.smart.py**
 - exercise5.clock.py
- OUTLINE
- TIMELINE

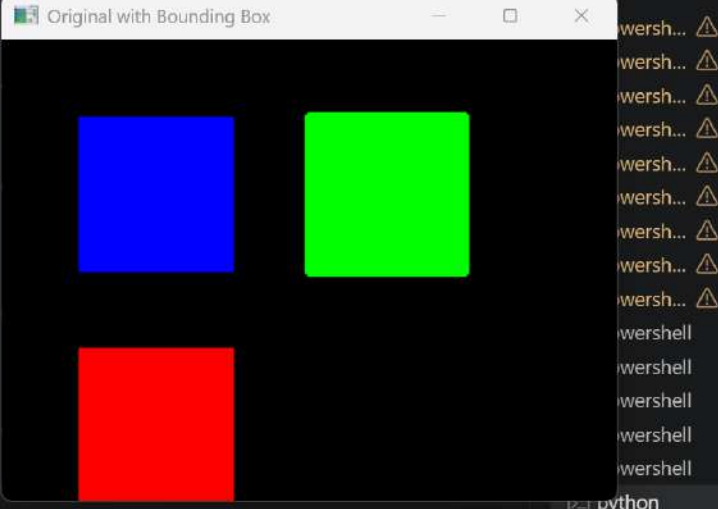
exercise4.smart.py

```
20 y_start = max(0, y - padding)
21 y_end = min(image.shape[0], y + h + padding)
22 x_start = max(0, x - padding)
23 x_end = min(image.shape[1], x + w + padding)
24
25 cropped = image[y_start:y_end, x_start:x_end]
26
27 # Draw bounding box on original
28 cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 3)
29
30 cv2.imshow("Original with Bounding Box", image)
31 cv2.imshow("Smart Cropped Object", cropped)
32 cv2.waitKey(0)
33 cv2.imwrite("smart_crop.png", cropped)
34 cv2.destroyAllWindows()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python exercise4.smart.py

Original with Bounding Box



File Edit Selection View Go Run Terminal Help python_test

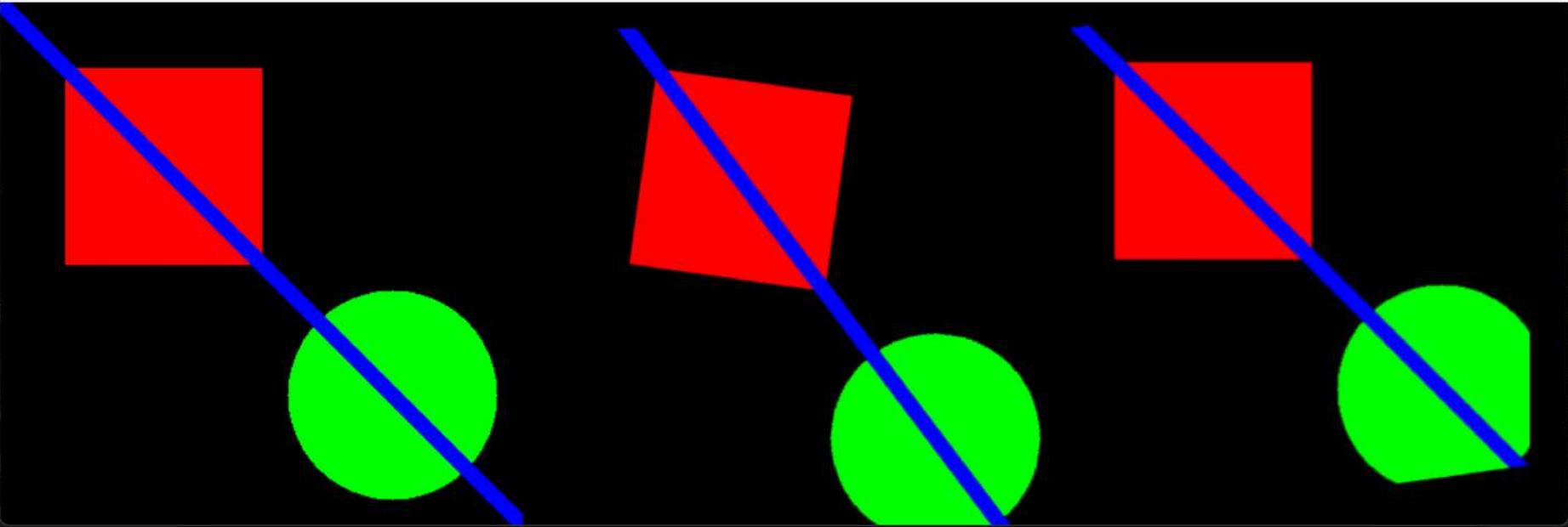
EXPLORER

- PYTHON_TEST
 - __pycache__
 - .vscode
 - settings.json
 - New folder
 - annotated_image.jpg
 - black.jpg
 - clock.png
 - create_test_image_day3.py
 - create_test_image.py
 - DAY-01 Program Outputs.pdf
 - DAY-02 Program outputs.pdf
 - DAY-03 Program Outputs.pdf
 - DAY-04 Program Outputs.pdf
 - day5_transformations.py
 - drawing_basics.py
 - drawing.py
 - exercise1.day3.py
 - exercise1.house.py
 - exercise1.py
 - exercise1.thumbnail.py
 - exercise2.day3.py
 - exercise2.panorama.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.animation.py
 - exercise3.chessboard.p
 - exercise3.day3.py
 - exercise3.py
 - exercise4.annotations.p
 - exercise4.day3.py
 - exercise4.smart.py
 - exercise5.alignment.py
- OUTLINE
- TIMELINE

Welcome exercise5.alignment.py

```
14 # Step 3: Find translation needed
15 # Step 4: Apply inverse transformations
16 # Solution approach (for learning)
17 print("CHALLENGE: Can you align the misaligned image?")
18 print("Hint: Try rotating by 8 degrees and translating by -30, -20")
19 # Try to fix it
20 fixed = misaligned.copy()
21 fixed = imutils.rotate(fixed, 8) # Counter-rotate
22 fixed = imutils.translate(fixed, -30, -20) # Counter-shift
23 # Compare
24 comparison = np.hstack([image, misaligned, fixed])
25 cv2.imshow("Original | Misaligned | Fixed", comparison)
26 cv2.waitKey(0)
27 cv2.imwrite("alignment_result.png", comparison)
28 cv2.destroyAllWindows()
```

Original | Misaligned | Fixed



python

Ln 28, Col 24 Spaces: 4 UTF-8 CRLF Python 3.14.5 Python 3.14 (64-bit)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - > __pycache__
 - > .vscode
 - settings.json
 - > New folder
 - annotated_image.jpg
 - black.jpg
 - clock.png
 - create_test_image_day3.py
 - create_test_image.py
 - crop.py
 - DAY-01 Program Outputs.pdf
 - DAY-02 Program outputs.pdf
 - DAY-03 Program Outputs.pdf
 - DAY-04 Program Outputs.pdf
 - day5_transformations.py
 - drawing_basics.py
 - drawing.py
 - exercise1.day3.py
 - exercise1.house.py
 - exercise1.py
 - exercise1.thumbnail.py
 - exercise2.day3.py
 - exercise2.panorama.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.animation.py
 - exercise3.chessboard.py
 - exercise3.day3.py
 - exercise3.py
 - exercise4.annotations.py
 - exercise4.day3.py
 - exercise4.smart.py
- OUTLINE
- TIMELINE

crop.py

```
1 # USAGE
2 # python crop.py --image ../images/trex.png
3
4 # Import the necessary packages
5 import numpy as np
6 import argparse
7 import cv2
8
9 # Construct the argument parser and parse the arguments
10 ap = argparse.ArgumentParser()
11 ap.add_argument("-i", "--image", required = True,
12                 help = "Path to the image")
13 args = vars(ap.parse_args())
14
15 # Load the image and show it
16 image = cv2.imread(args["image"])
17 cv2.imshow("Original", image)
18
19 # Cropping an image is as simple as using array slices
20 # in NumPy! Let's crop out the face of the T-Rex. The
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python crop.py --image terex.png



Ln 27, Col 15 Spaces: 4 UTF-8 CRLF Python 3.14.5 Python 3.14 (64-bit)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - exercise1.house.py
 - exercise1.py
 - exercise1.thumbnail.py
 - exercise2.day3.py
 - exercise2.panorama.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.animation.py
 - exercise3.chessboard.py
 - exercise3.day3.py
 - exercise3.py
 - exercise4.annotations.py
 - exercise4.day3.py
 - exercise4.smart.py
 - exercise5.alignment.py
 - exercise5.clock.py
 - flipping.py
 - getting_and_setting.py
 - gradient.png
 - house.png
 - image.py
 - imutils.py
 - load_display_save.py
 - my_masterpiece.png
 - output.bmp
 - output.jpg
 - output.tiff
 - panorama_prep.png
 - pixel_basics.py
 - red.jpg
 - saved_image.jpg
 - saved_image.png
 - smart_crop.png
- OUTLINE
- TIMELINE

flipping.py X

```
15 image = cv2.imread(args["image"])
16 cv2.imshow("Original", image)
17
18 # Flip the image horizontally
19 flipped = cv2.flip(image, 1)
20 cv2.imshow("Flipped Horizontally", flipped)
21
22 # Flip the image vertically
23 flipped = cv2.flip(image, 0)
24 cv2.imshow("Flipped Vertically", flipped)
25
26 # Flip the image along both axes
27 flipped = cv2.flip(image, -1)
28 cv2.imshow("Flipped Horizontally & Vertically", flipped)
29 cv2.waitKey(0)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\raghu\OneDrive\Desktop\python_test> python flipping.py --image terex.png
```

Flipped Horizontally



Flipped Vertically



Original



Flipped Horizontally & Vertically



Ln 29, Col 15 Spaces: 4 UTF-8 CRLF Python 3.14.5 Python 3.14 (64-bit)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - translation.py
 - exercise3.py
 - exercise4.annotations.py
 - exercise4.day3.py
 - exercise4.smart.py
 - exercise5.alignment.py
 - exercise5.clock.py
 - flipping.py
 - getting_and_setting.py
 - gradient.png
 - house.png
 - image.py
 - imutils.py
 - load_display_save.py
 - my_masterpiece.png
 - output.bmp
 - output.jpg
 - output.tiff
 - panorama_prep.png
 - pixel_basics.py
 - red.jpg
 - saved_image.jpg
 - saved_image.png
 - smart_crop.png
 - target.png
 - terex.png
 - test_gradient.png
 - test_image_gray.png
 - test_image.png
 - test_pattern.png
 - thumbnail_gallery.py
 - translation.py
 - verify.py

translation.py

```
1 import imutils
2 import cv2
3
4 # Construct the argument parser and parse the arguments
5 ap = argparse.ArgumentParser()
6 ap.add_argument("-i", "--image", required = True,
7                 help = "Path to the image")
8 args = vars(ap.parse_args())
9
10 # Load the image and show it
11 image = cv2.imread(args["image"])
12 cv2.imshow("Original", image)
13
14 # NOTE: Translating (shifting) an image is given by a
15 # NumPy matrix in the form:
16 # [[1, 0, shiftX], [0, 1, shiftY]]
17 # You simply need to specify how many pixels you want
18 # to shift the image in the X and Y direction.
19 # Let's translate the image 25 pixels to the right and
20 # 50 pixels down
21 M = np.float32([[1, 0, 25], [0, 1, 50]])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python translation.py --image terex.png

Shifted Down



Original



Shifted Down and Right



Shifted Up and Left



UTF-8 CRLF Python 3.14.5 Python 3.14 (64-bit)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - exercise4.day3.py
 - exercise4.smart.py
 - exercise5.alignment.py
 - exercise5.clock.py
 - flipping.py
 - getting_and_setting.py
 - gradient.png
 - house.png
 - image.py
 - imutils.py
 - load_display_save.py
 - my_masterpiece.png
 - output.bmp
 - output.jpg
 - output.tiff
 - panorama_prep.png
 - pixel_basics.py
 - red.jpg
 - rotate.py
 - saved_image.jpg
 - saved_image.png
 - smart_crop.png
 - target.png
 - terex.png
 - test_gradient.png
 - test_image_gray.png
 - test_image.png
 - test_pattern.png
 - thumbnail_gallery.png
 - translation.py
 - verify.py
 - white.jpg

rotate.py

```
1 import numpy as np
2 import argparse
3 import imutils
4 import cv2
5
6 # Construct the argument parser and parse the arguments
7 ap = argparse.ArgumentParser()
8 ap.add_argument("-i", "--image", required = True,
9                 help = "Path to the image")
10 args = vars(ap.parse_args())
11
12 # Load the image and show it
13 image = cv2.imread(args["image"])
14 cv2.imshow("Original", image)
15
16 # Grab the dimensions of the image and calculate the center
17 # of the image
18 (h, w) = image.shape[:2]
19 center = (w // 2, h // 2)
20
21 # Rotate our image by 45 degrees
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python rotate.py --image terex.png

Rotated by 45 Degrees

Rotated by 180 Degrees

Rotated by -90 Degrees

Original

Ln 39, Col 15 Spaces: 4 UTF-8 CRLF Python 3.14.5 Python 3.14 (64-bit)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

PYTHON_TEST

- exercise4.day3.py
- exercise4.smart.py
- exercise5.alignment.py
- exercise5.clock.py
- flipping.py
- getting_and_setting.py
- gradient.png
- house.png
- image.py
- imutils.py
- load_display_save.py
- my_masterpiece.png
- output.bmp
- output.jpg
- output.tiff
- panorama_prep.png
- pixel_basics.py
- red.jpg
- resize.py
- rotate.py
- saved_image.jpg
- saved_image.png
- smart_crop.png
- target.png
- terex.png
- test_gradient.png
- test_image_gray.png
- test_image.png
- test_pattern.png
- thumbnail_gallery.png
- translation.py
- verify.py

resize.py > ...

```
1 # USAGE
2 # python resize.py --image ../images/trex.png
3
4 # Import the necessary packages
5 import numpy as np
6 import argparse
7 import imutils
8 import cv2
9
10 # Construct the argument parser and parse the arguments
11 ap = argparse.ArgumentParser()
12 ap.add_argument("-i", "--image", required = True,
13                 help = "Path to the image")
14 args = vars(ap.parse_args())
15
16 # Load the image and show it
17 image = cv2.imread(args["image"])
18 cv2.imshow("Original", image)
19
20 # We need to keep in mind aspect ratio so the image does
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python resize.py --image terex.png

Original

